

1 Introduction to Java

Java is a high-level, object-oriented, platform-independent programming language developed by Sun Microsystems (now owned by Oracle) in 1995. Its design philosophy emphasizes portability, robustness, and simplicity, with the slogan “**Write Once, Run Anywhere**” (**WORA**). Java achieves this through the Java Virtual Machine (JVM), which allows compiled Java code (bytecode) to run on any device with a JVM, regardless of the underlying hardware or operating system.

1.1 Key Features of Java

- **Platform Independence:** Java code is compiled into bytecode, which runs on the JVM, making it portable across platforms.
- **Object-Oriented:** Built around objects and classes, supporting principles like encapsulation, inheritance, polymorphism, and abstraction.
- **Robust:** Strong type-checking, exception handling, and automatic memory management (garbage collection) minimize errors.
- **Secure:** Features like the sandbox environment and bytecode verification ensure secure execution.
- **Multithreaded:** Built-in support for concurrent programming.
- **High Performance:** Just-In-Time (JIT) compilation optimizes bytecode execution.
- **Rich API:** Extensive standard libraries for networking, I/O, data structures, and more.
- **Community and Ecosystem:** Large community, frameworks (e.g., Spring, Hibernate), and tools (e.g., Maven, Gradle).

1.2 Use Cases

- Web development (e.g., Spring Boot)
- Mobile apps (Android development)
- Enterprise applications (e.g., banking systems)
- Big data (e.g., Apache Hadoop)
- Scientific applications
- Game development (e.g., using LibGDX)

2 Setting Up Java

To start coding in Java, you need:

1. **Java Development Kit (JDK):** Includes the compiler (`javac`), JVM, and standard libraries. Download the latest version (e.g., JDK 21 as of 2025) from Oracle or adopt OpenJDK.
2. **Integrated Development Environment (IDE):** Popular choices include IntelliJ IDEA, Eclipse, or NetBeans for coding, debugging, and project management.

3. Environment Configuration:

- Set the `JAVA_HOME` environment variable to the JDK installation path.
- Add the JDK's `bin` directory to your system's `PATH` to use `java` and `javac` commands.

To verify installation:

```
1 java -version
2 javac -version
```

3 Java Syntax Basics

Java programs are written in plain text files with a `.java` extension and compiled into `.class` files (bytecode) using the `javac` compiler. Here's a simple "Hello, World!" program:

```
1 public class HelloWorld {
2     public static void main(String[] args) {
3         System.out.println("Hello, World!");
4     }
5 }
```

3.1 Key Elements

- **Class:** Java code is organized in classes. The `public class` name must match the file name (e.g., `HelloWorld.java`).
- **Main Method:** `public static void main(String[] args)` is the entry point of a Java program.
- **Output:** `System.out.println()` prints to the console.
- **Case Sensitivity:** Java is case-sensitive (`Main` $\neq$ `main`).
- **Semicolons:** Statements end with `;`.

3.2 Basic Syntax Rules

- **Comments:**

```
1 // Single-line comment
2 /* Multi-line comment */
3 /** Javadoc comment */
```

- **Identifiers:** Names for classes, methods, and variables (e.g., `myVariable`). Must start with a letter, `_`, or `$`, and cannot be a reserved keyword (e.g., `class`, `int`).
- **Keywords:** Reserved words like `public`, `static`, `void`, `if`, `for`, etc.

4 Data Types and Variables

Java is **statically typed**, meaning variables must be declared with a type before use.

Type	Size	Description	Example
byte	1 byte	8-bit integer (-128 to 127)	byte b = 100;
short	2 bytes	16-bit integer	short s = 1000;
int	4 bytes	32-bit integer	int i = 12345;
long	8 bytes	64-bit integer	long l = 123L;
float	4 bytes	32-bit floating-point	float f = 3.14f;
double	8 bytes	64-bit floating-point	double d = 3.14;
char	2 bytes	Single Unicode character	char c = 'A';
boolean	1 bit	True or false	boolean b = true;

4.1 Primitive Data Types

4.2 Non-Primitive (Reference) Types

- **String:** Represents a sequence of characters (e.g., `String name = "Alice";`).
- **Arrays:** Collections of elements of the same type (e.g., `int[] numbers = {1, 2, 3};`).
- **Classes/Objects:** User-defined types (covered in OOP section).

4.3 Variable Declaration and Scope

- **Local Variables:** Declared inside methods, only accessible within that method.
- **Instance Variables:** Declared in a class, accessible to all methods of the class.
- **Static Variables:** Belong to the class, shared across all instances (use `static` keyword).

```

1 public class Example {
2     int instanceVar = 10; // Instance variable
3     static int staticVar = 20; // Static variable
4     void method() {
5         int localVar = 30; // Local variable
6     }
7 }

```

5 Operators

Java supports standard operators for arithmetic, comparison, logical operations, and more:

- **Arithmetic:** `+`, `-`, `*`, `/`, `%` (modulus), `++`, `--`
- **Comparison:** `==`, `!=`, `<`, `>`, `<=`, `>=`
- **Logical:** `&&` (AND), `||` (OR), `!` (NOT)
- **Assignment:** `=`, `+=`, `-=`, `*=`, `/=`, etc.
- **Bitwise:** `&`, `|`, `^`, `~`, `<<`, `>>`, `>>>`
- **Ternary:** `condition ? valueIfTrue : valueIfFalse`

Example:

```

1 int a = 10, b = 5;
2 int sum = a + b; // 15
3 boolean isEqual = (a == b); // false
4 int result = (a > b) ? a : b; // 10

```

6 Control Flow

Java provides constructs to control program execution.

6.1 Conditional Statements

- **if-else:**

```

1 int x = 10;
2 if (x > 0) {
3     System.out.println("Positive");
4 } else if (x < 0) {
5     System.out.println("Negative");
6 } else {
7     System.out.println("Zero");
8 }

```

- **switch** (enhanced in Java 14+):

```

1 int day = 2;
2 switch (day) {
3     case 1 -> System.out.println("Monday");
4     case 2 -> System.out.println("Tuesday");
5     default -> System.out.println("Invalid");
6 }

```

6.2 Loops

- **for:**

```

1 for (int i = 0; i < 5; i++) {
2     System.out.println(i); // Prints 0 to 4
3 }

```

- **Enhanced for** (for-each):

```

1 int[] arr = {1, 2, 3};
2 for (int num : arr) {
3     System.out.println(num);
4 }

```

- **while:**

```

1 int i = 0;
2 while (i < 5) {
3     System.out.println(i);
4     i++;
5 }

```

- **do-while:**

```

1  int i = 0;
2  do {
3      System.out.println(i);
4      i++;
5  } while (i < 5);

```

6.3 Break and Continue

- **break:** Exits a loop or switch.
- **continue:** Skips the current iteration and continues with the next.

7 Object-Oriented Programming (OOP)

Java is fundamentally object-oriented, built around four core principles: **encapsulation**, **inheritance**, **polymorphism**, and **abstraction**.

7.1 Classes and Objects

- **Class:** A blueprint for objects.
- **Object:** An instance of a class.

```

1  public class Dog {
2      String name; // Instance variable
3      int age;
4
5      // Constructor
6      public Dog(String name, int age) {
7          this.name = name;
8          this.age = age;
9      }
10
11     // Method
12     public void bark() {
13         System.out.println(name + " says Woof!");
14     }
15 }
16
17 public class Main {
18     public static void main(String[] args) {
19         Dog dog = new Dog("Buddy", 3); // Creating an object
20         dog.bark(); // Output: Buddy says Woof!
21     }
22 }

```

7.2 Encapsulation

- Bundling data (fields) and methods, restricting direct access to fields using access modifiers (**private**, **protected**, **public**).
- Use getters and setters:

```

1 public class Person {
2     private String name;
3
4     public String getName() {
5         return name;
6     }
7
8     public void setName(String name) {
9         this.name = name;
10    }
11 }

```

7.3 Inheritance

- Allows a class to inherit properties and methods from another using `extends`.

```

1 class Animal {
2     void eat() {
3         System.out.println("This animal eats food.");
4     }
5 }
6
7 class Dog extends Animal {
8     void bark() {
9         System.out.println("Woof!");
10    }
11 }
12
13 public class Main {
14     public static void main(String[] args) {
15         Dog dog = new Dog();
16         dog.eat(); // Inherited method
17         dog.bark(); // Dog-specific method
18     }
19 }

```

7.4 Polymorphism

- Allows objects to be treated as instances of their parent class (method overriding or overloading).

```

1 class Animal {
2     void sound() {
3         System.out.println("Some sound");
4     }
5 }
6
7 class Cat extends Animal {
8     @Override
9     void sound() {
10        System.out.println("Meow");
11    }
12 }
13
14 public class Main {
15     public static void main(String[] args) {

```

```

16         Animal myCat = new Cat(); // Polymorphism
17         myCat.sound(); // Output: Meow
18     }
19 }

```

7.5 Abstraction

- Hiding implementation details using abstract classes or interfaces.

- **Abstract Class:**

```

1 abstract class Shape {
2     abstract void draw();
3 }
4
5 class Circle extends Shape {
6     void draw() {
7         System.out.println("Drawing a circle");
8     }
9 }

```

- **Interface** (defines methods without implementation):

```

1 interface Drawable {
2     void draw();
3 }
4
5 class Rectangle implements Drawable {
6     public void draw() {
7         System.out.println("Drawing a rectangle");
8     }
9 }

```

8 Exception Handling

Java uses exceptions to handle errors gracefully.

- **try-catch:**

```

1 try {
2     int result = 10 / 0; // Throws ArithmeticException
3 } catch (ArithmeticException e) {
4     System.out.println("Error: " + e.getMessage());
5 } finally {
6     System.out.println("This always executes");
7 }

```

- **Throwing Exceptions:**

```

1 public void checkAge(int age) throws IllegalArgumentException {
2     if (age < 18) {
3         throw new IllegalArgumentException("Age must be 18 or older");
4     }
5 }

```

- **Custom Exceptions:**

```
1 class CustomException extends Exception {
2     public CustomException(String message) {
3         super(message);
4     }
5 }
```

9 Collections Framework

The Java Collections Framework provides data structures to store and manipulate groups of objects.

- **Key Interfaces:**

- **List:** Ordered, allows duplicates (e.g., `ArrayList`, `LinkedList`).
- **Set:** No duplicates (e.g., `HashSet`, `TreeSet`).
- **Map:** Key-value pairs (e.g., `HashMap`, `TreeMap`).

- **Example:**

```
1 import java.util.ArrayList;
2 import java.util.HashMap;
3
4 public class Main {
5     public static void main(String[] args) {
6         // List
7         ArrayList<String> list = new ArrayList<>();
8         list.add("Apple");
9         list.add("Banana");
10        System.out.println(list); // [Apple, Banana]
11
12        // Map
13        HashMap<String, Integer> map = new HashMap<>();
14        map.put("Alice", 25);
15        map.put("Bob", 30);
16        System.out.println(map.get("Alice")); // 25
17    }
18 }
```

10 Multithreading

Java supports concurrent programming through threads.

- **Creating a Thread:**

- Extend Thread class:

```
1 class MyThread extends Thread {
2     public void run() {
3         System.out.println("Thread running");
4     }
5 }
6
7 public class Main {
8     public static void main(String[] args) {
```



```

9         MyThread thread = new MyThread();
10        thread.start(); // Starts the thread
11    }
12}

```

– Implement Runnable interface:

```

1 class MyRunnable implements Runnable {
2     public void run() {
3         System.out.println("Runnable running");
4     }
5 }
6
7 public class Main {
8     public static void main(String[] args) {
9         Thread thread = new Thread(new MyRunnable());
10        thread.start();
11    }
12}

```

- **Synchronization:** Use synchronized to prevent thread interference:

```

1 public synchronized void safeMethod() {
2     // Thread-safe code
3 }

```

11 Java I/O

Java provides APIs for file and network operations.

- **Reading a File:**

```

1 import java.io.BufferedReader;
2 import java.io.FileReader;
3
4 public class Main {
5     public static void main(String[] args) throws Exception {
6         BufferedReader reader = new BufferedReader(new FileReader("
7             file.txt"));
8         String line;
9         while ((line = reader.readLine()) != null) {
10            System.out.println(line);
11        }
12        reader.close();
13    }
14}

```

- **Writing to a File:**

```

1 import java.io.FileWriter;
2
3 public class Main {
4     public static void main(String[] args) throws Exception {
5         FileWriter writer = new FileWriter("output.txt");
6         writer.write("Hello, Java!");
7         writer.close();
8     }
9 }

```

12 Java 8+ Features

Modern Java versions introduced significant improvements:

- **Lambda Expressions:** Concise syntax for functional programming.

```
1 (a, b) -> a + b // Lambda for adding two numbers
```

- **Stream API:** Process collections functionally.

```
1 import java.util.Arrays;
2 import java.util.List;
3
4 public class Main {
5     public static void main(String[] args) {
6         List<Integer> numbers = Arrays.asList(1, 2, 3, 4);
7         numbers.stream()
8             .filter(n -> n % 2 == 0)
9             .forEach(System.out::println); // Prints 2, 4
10    }
11 }
```

- **Optional:** Handles null values safely.

```
1 import java.util.Optional;
2
3 Optional<String> name = Optional.ofNullable(null);
4 System.out.println(name.orElse("Default")); // Prints Default
```

- **Records (Java 16+):** Immutable data classes.

```
1 record Person(String name, int age) {}
```

13 Java Ecosystem

- **Build Tools:**

- **Maven:** Manages dependencies and builds (uses `pom.xml`).
- **Gradle:** Flexible build tool (uses `build.gradle`).

- **Frameworks:**

- **Spring:** For enterprise apps, web services, and microservices.
- **Hibernate:** For database ORM (Object-Relational Mapping).
- **JavaFX:** For desktop GUI applications.

- **Testing:**

- **JUnit:** For unit testing.
- **Mockito:** For mocking dependencies.

- **Java EE (Jakarta EE):** For enterprise applications (e.g., servlets, JSP).

14 Best Practices

- **Code Organization:** Use packages to organize classes (e.g., `com.example.myapp`).
- **Naming Conventions:**
 - Classes: `CamelCase` (e.g., `MyClass`).
 - Methods/Variables: `camelCase` (e.g., `myMethod`).
 - Constants: `UPPER_CASE` (e.g., `MAX_VALUE`).
- **Error Handling:** Always handle exceptions appropriately.
- **Documentation:** Use Javadoc for clear documentation.
- **Modularity:** Break code into reusable methods and classes.